FORM notes

Ana Costa Pereira

26/06/2026

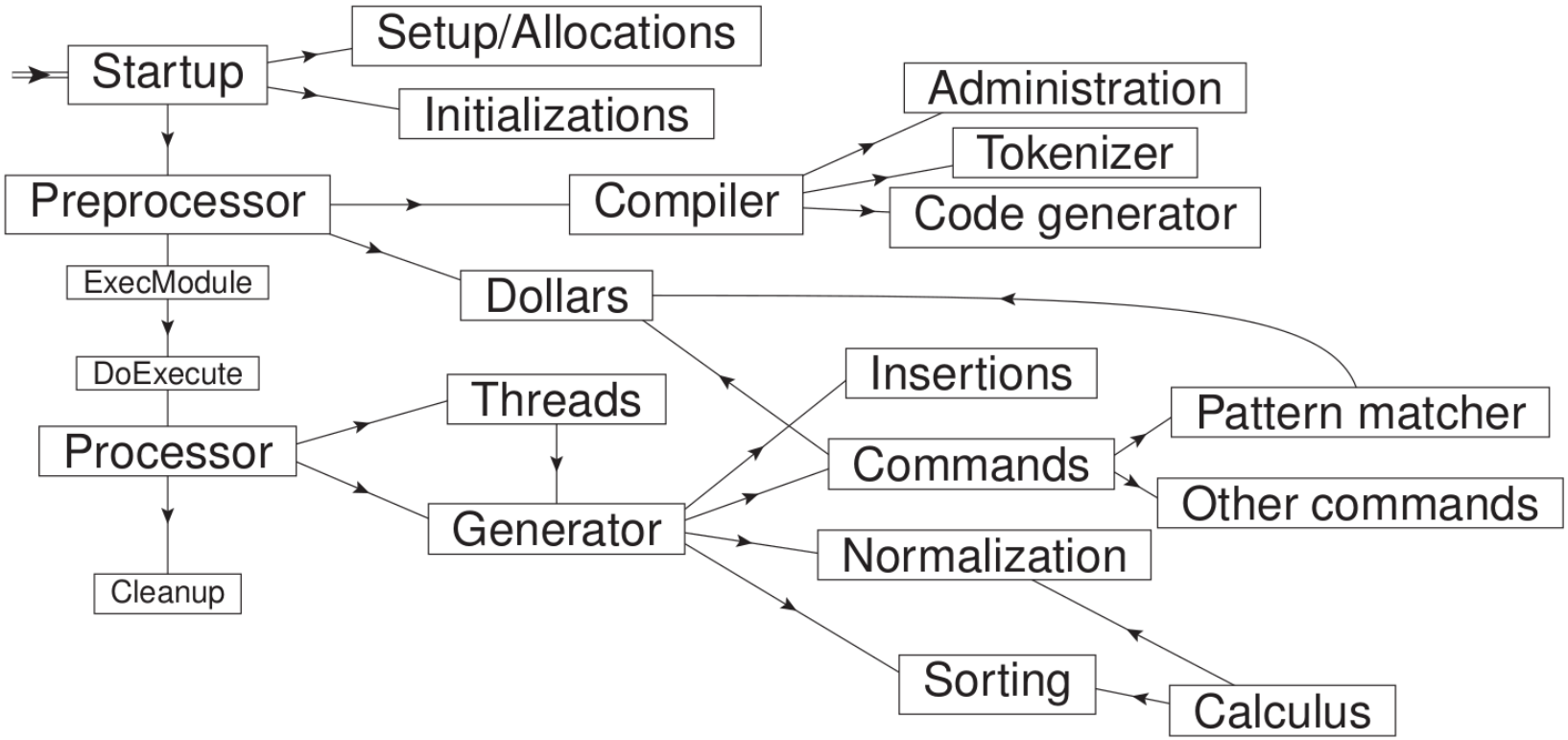
FORM

- Written in C language (some parts C++)
- Focous on execution speed rather then development speed;
- Sourced files: header files (declarations), C and C++ files;
- GUI that allows code folding (including fold nesting);

FORM sources

- each .c or .cc source file there is such a fold and it calls the necessary include files and contains local variables;
- form3.h contains calls to most of the .h files
- global variables are contained inside just a few data structs that are defined in the file structs.h

Layout


Startup
Setup/Allocations
Initializations
Preprocessor
Compiler
Administration
Tokenizer
Code generator
ExecModule
Dollars
DoExecute
Threads
Processor
Insertions
Pattern matcher
Commands
Other commands
Generator
Normalization
Cleanup
Sorting
Calculus

- **Expression:** sequence of terms;
- **Term:** sequence of numbers of type WORD;
- Single linear buffer that contains all the terms of the expression and the expression is the sum of all its terms;

Sorting

- Global variables, buffers and tracking data reside in single master structure A.
- TFORM runs on multi-core processors POSIX threads (pthreads);
- If each threads could access and change the glovab memory, it would clash and colapse;
- Wach threads gets allocated a private copy of A with a pointer, which is $*B$.
- When compiling for parallel execution, private sub-structures N, R, T are decoupled from A and placed into teh threads local $*B$;
- To incorporate FORM and TFORM in the same code source - using Macro AT that gets replaces by the correct option depending on which version the user is running.

Sorting

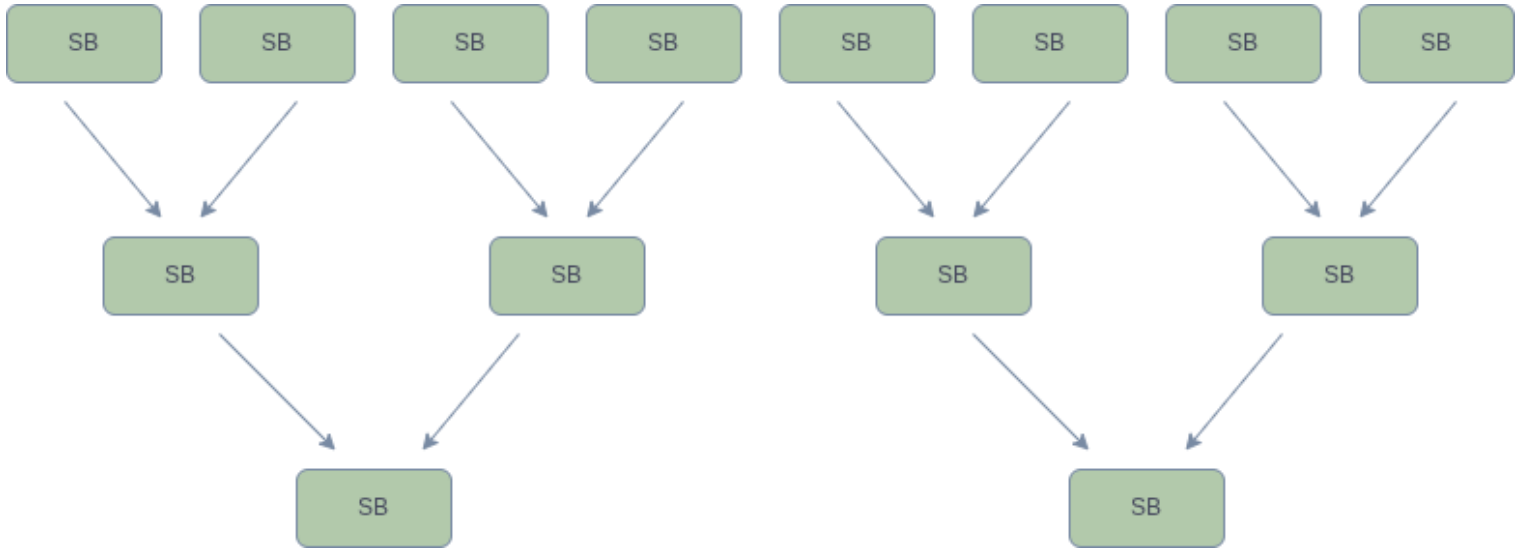
- Each worker has a local copy of $*B$ with private data and shared memory;
- Master makes allocations
- N workers which get tasks assigned by the master and give the output back to the master once they finish;
- $N - 2$ sortbots that help the sorting

Sorting

- Each worker performs a local sort of its generated terms.
- Workers produce independent sorted streams.
- Sortbots merge pairs of sorted outputs in parallel.
- The master performs only the final merge and writes output.
- Goal: reduce master-thread bottlenecks and maximize worker utilization.

- Multi-threads: what to do when multiple threads try to update a global variable at the same time?
- **Lock routine**: worker has to request a lock both for the reader and the writing and souble check after getting the lock if it wants to update the variable, and only then it can update and the locks are released.

Sorting



- Master makes the allocations for the workers;
- For each N workers, $N - 2$ SB are created;

When a worker runs algebraic expression (example id statement), generated a stream of output terms - there is no rearrengement or combination of terms - they are packet together on a small buffer one after the other to maximize speedVarious stages in which sorting takes place (once filled up moves to the next):1 short buffer: routine **SplitMerge**;2 large buffer (large allocation RAM buffer)3 sort file (disk spill stage)

After we merge results from various cores

Sorting - small buffer

- Moving large algebraic terms across the memory is computationally expensive; instead FORM creates an array of memory pointers - each pointer points to a start of a term;
- **SplitMerge** evaluated the terms to which the pointers point and sorts the pointers;
- Identical terms are now consecutively next to each other in memory, where coefficients can combine and cancellations may occur;
- In the combination of consecutive terms during sorting it may happend that the term that results out of this takes more space in memory compared to the seperate terms before the combination: the tail of the buffer contains a memory pad where this terms are written
- Once all terms are sorted they are copied to the Large Buffer and the small Buffer is wiped clean to receive the next batch of terms;

TFORM code lives mostly in threads.c. Inner workings:

- Features that deal with the identities of the threads: **BeginIdentities**;
- Given the number of threads as input, determine how many workers we have;
- Allocations are made for the workers: **StartAllThreads**;

- Memory is shared globally between all threads;
- Avoid overwritting: each thread makes a private copy of the data: allocation of 1/N of the master's buffer size for each thread and sortbot
- After buffers are set up, workers are ready to start working;
- Master reads input expressions from the scratch file and distribute terms for workers;
- If master sends terms one by one takes too long;
- Instead master sends buckets, $1 bucket = 500 terms$ - done by **ThreadsProcessor**;

- Workers receive the buckets by the master and they are finished when the buckets are empty;
- If a worker is completely done and there are no more buckets to be distributed, the master moves buckets from workers that are still processing terms to these idle workers: **RunThread**;

TFORM

- After processing the terms, the threads will sort;
- Each worker has: small buffer, large buffer, sort file, stage4 if necessary;
- During this final merge we use sortbots - merge each two workers into a single output, until there is only 2 sortbots left;
- Then the master merge the final two outputs and writes into the output scratch file
- Two major bottlenecks in TFORM: reading of the input with division of terms and the inverse tree at the end with the writing to the file by the master;
- Ways to improve performance: Use **Brackets** option: all the terms in the bracket will end up in the same worker, so cancellations can be done earlier in the sorting (rather then terms that could cancel ending up in different works and therefore the cancellation would happend only in later stages of the sorting);

Best setup for FORM and TFORM

- Large memory since disks can slow things down for large expressions;

Working of the code:

- Terms are allowed to overlap between blocks;
- Reading thread always locks pair of locks - the current thread and the previous thread;
- 0th block - the overlap in the end of term is copied to 0th so that the terms are contiguous - this is no longer needed when we impose that only full terms can fit into the blocks

Before changes

- Default buffer sizes were increased over time
- Size of sortbots increased;
- The output from a worker fits in a single block $\longrightarrow$ no advantage on running in parallel;
- Each stage of sortbot merge tree runs one after the other
- Terms can be partially shared between blocks: when a term is being written in a block and the block finishes, the term keeps being written into the next block

After changes

- Blocks are made smaller
- Whole terms can only be written in one block: if it does not fit, it goes into the next block
- Workers that fill the blocks try to detect if there is a reader waiting for them
- Obtain a lock of the block before filling the block; if not, it means the reading thread already has the lock which means that it is processing the block which means it wants small terms from the current filling block, so we move on to the next block in writing thread
- There is always a minimum number of terms in the block - MINIMUMNUMBEROFTERMS

Some results after the changes

- No decrease in performance for any tests;
- Performance improvement for benchmarks where term merging is expensive (PolyRatFun, PolyRatFunExpand)
- Forcer, mbox1l